

CHAPTER 7: SOFTWARE SECURITY

Le Thi Bao Thu

Faculty of Computer Science & Engineering

HCMC University of Technology

thule@hcmut.edu.vn

(Adapted from the slides of Information Security: Principles and Practice, 3rd edition, Mark Stamp, 2021)

Content

Software Security Introduction

Program Flaws

Malware

Miscellaneous Software-Based Attacks

Software Reverse Engineering (SRE)

Digital Rights Management

Secure Software Development

SOFTWARE SECURITY

INTRODUCTION

Why Software?

- ❑ Why is software as important to security as crypto, access control, protocols?
- ❑ Virtually all information security features are implemented in software
- ❑ If your software is subject to attack, your security can be broken
 - Regardless of strength of crypto, access control, or protocols
- ❑ Software is a poor ***foundation*** for security

Secure Software

- ❑ In software engineering, try to ensure that a program does what is intended
- ❑ **Secure** software engineering requires that software **does what is intended...**
- ❑ **...and nothing more**

Complexity

- ❑ “Complexity is the enemy of security”, Paul Kocher, Cryptography Research, Inc.

System	Lines of Code (LOC)
Netscape	17 million
Space Shuttle	10 million
Linux kernel 2.6.0	5 million
Windows XP	40 million
Mac OS X 10.4	86 million
Boeing 777	7 million

Lines of Code and Bugs

❑ Conservative estimate: 5 bugs/10,000 LOC

❑ **Do the math**

- Typical computer: 3k exe's of 100k LOC each
- Conservative estimate: 50 bugs/exe
- Implies about 150k bugs per computer
- So, 30,000-node network has 4.5 billion bugs
- Maybe only 10% of bugs security-critical and only 10% of those remotely exploitable
- Then “only” 45 million critical security flaws!

Software Security Topics

❑ Program flaws (unintentional)

- Buffer overflow
- Race conditions

❑ Malicious software (intentional)

- Viruses
- Worms
- Other breeds of malware

PROGRAM FLAWS

Buffer overflow - Attack Scenario

- ❑ Users enter data into a Web form
- ❑ Web form is sent to server
- ❑ Server writes data to array called buffer, without checking length of input data
- ❑ Data “overflows” buffer
 - Such overflow might enable an attack
 - If so, attack could be carried out by anyone with Internet access

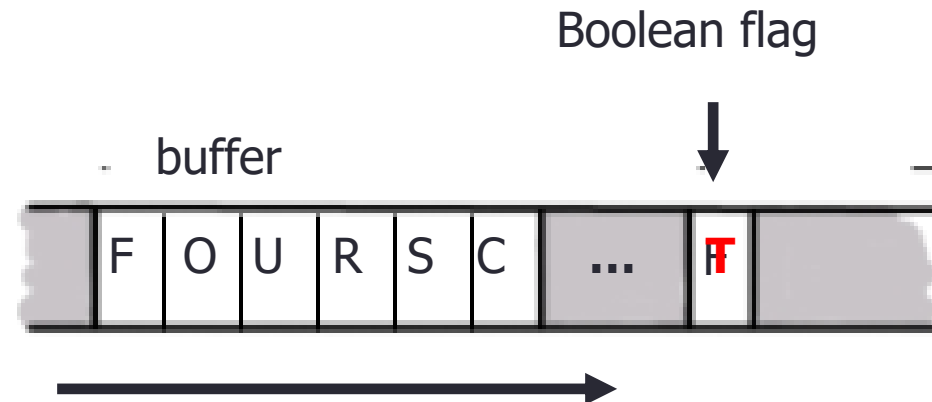
Buffer Overflow

```
int main() {  
    int buffer[10];  
    buffer[20] = 37; }
```

- ❑ **Q:** What happens when code is executed?
- ❑ **A:** Depending on what resides in memory at location “buffer[20]”
 - Might overwrite **user** data or code
 - Might overwrite **system** data or code
 - Or program could work just fine

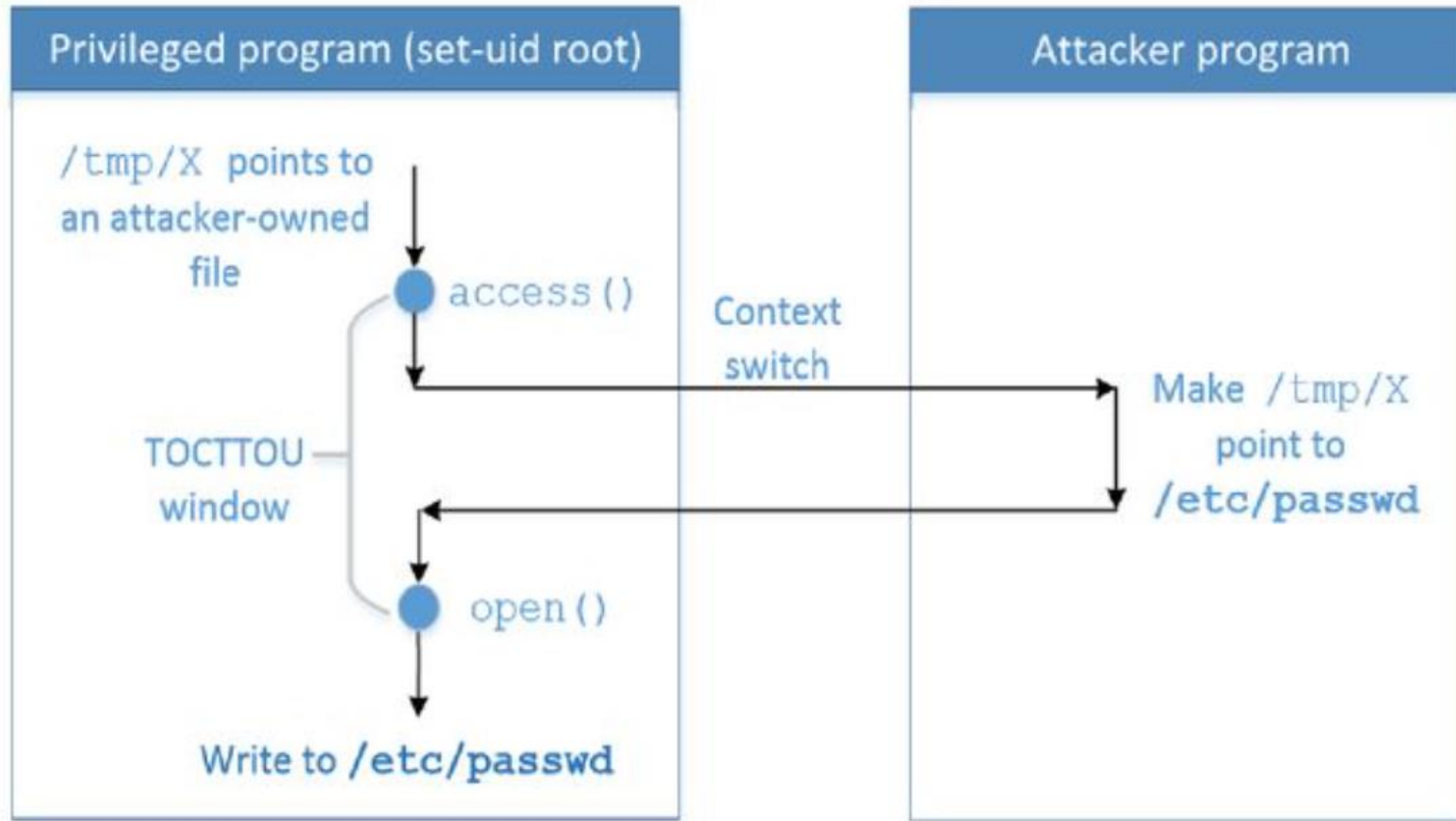
Simple Buffer Overflow

- ❑ Consider boolean flag for authentication
- ❑ Buffer overflow could overwrite flag allowing anyone to authenticate



Race conditions

- ❑ Also known as Time-Of-Check To Time-Of-Use (TOCTTOU)



To win the race condition (TOCTTOU window), we need two processes :

- Run vulnerable program in a loop
- Run the attack program

MALWARE

Malicious Software

- ❑ Types of malware (no standard definition)
 - **Virus** — passive propagation
 - **Worm** — active propagation
 - Trojan horse — unexpected functionality
 - Trapdoor/backdoor — unauthorized access
 - Rabbit — exhaust system resources
 - Spyware — steals info, such as passwords
 - Botnets — malware for hire
 - Ransomware — Malware encrypts user data

Malicious Software

❑ Types of malware (no standard definition):

- Virus: passive propagation
- Worm: active propagation
- Trojan horse: unexpected functionality
- Trapdoor/backdoor: unauthorized access
- Rabbit: exhaust system resources
- Spyware: steals info, such as passwords
- Botnets: malware for hire
- Ransomware: Malware encrypts user data

Trojan Horse Example

- ❑ Trojan: unexpected functionality
- ❑ Prototype trojan for the Mac
- ❑ File icon for freeMusic.mp3:

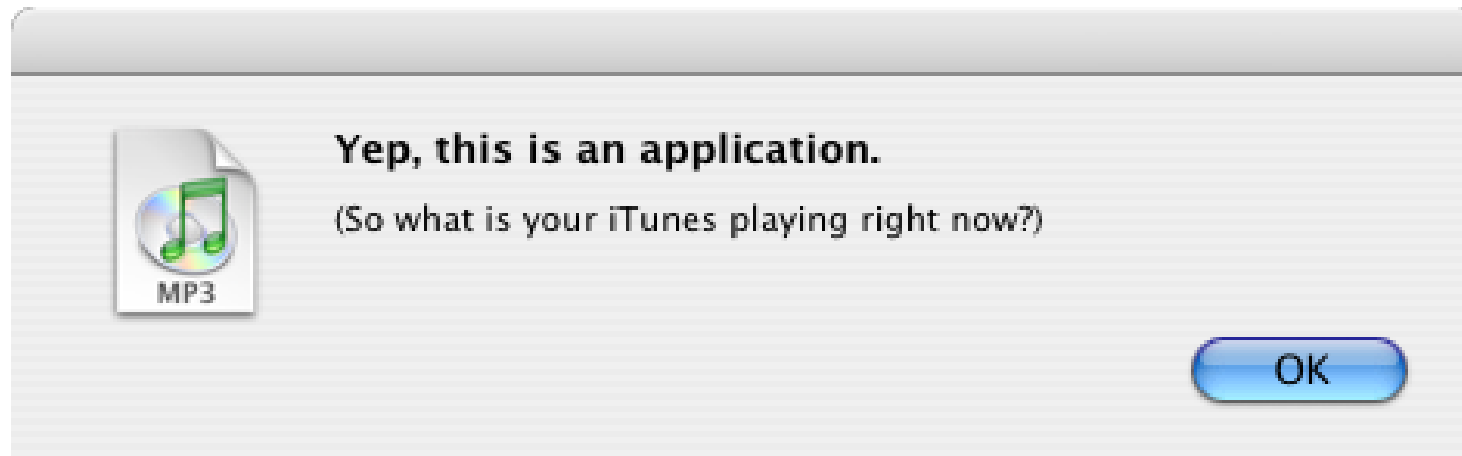


- ❑ For a real mp3, double click on icon
 - iTunes opens
 - Music in mp3 file plays
- ❑ But for freeMusic.mp3, unexpected results...

freeMusic.mp3

Mac Trojan

- ❑ Double click on freeMusic.mp3
 - iTunes opens (expected)
 - “Wild Laugh” (not expected)
 - Message box (not expected)



Trojan Example

- ❑ How does freeMusic.mp3 trojan work?
- ❑ This “mp3” is an application, not data



- ❑ This trojan is harmless, but...
- ❑ ...could have done anything user could do
 - Delete files, download files, launch apps, etc.

Botnet

- ❑ Botnet: a “network” of infected machines
- ❑ Infected machines are “bots”
 - Victim is unaware of infection (stealthy)
- ❑ Botmaster controls botnet

- ❑ Botnets used for...
 - Spam, DoS attacks, keylogging, ID theft, etc.

Botnet Examples

❑ Mariposa

- Used to steal credit card info
- Creator arrested in July 2010

❑ Conficker

- Estimated 10M infected hosts (2009)

❑ Kraken

- Largest as of 2008 (400,000 infections)

❑ Srizbi

- For spam, one of largest as of 2008

Ransomware

- ❑ Recently, ransomware very popular
- ❑ Ransomware encrypts important info
 - Key may be provided if ransom paid
 - Sometimes key works, sometimes not...
- ❑ To pay or not to pay?
 - Recent case, reportedly paid \$5M
 - In 2020 total ransom estimated \$350M
 - Payments usually in cryptocurrency

CryptoLocker Ransomware

- ❑ CryptoLocker (2013, Russian origin)
 - Email with attached “pdf” (actually an exe)
- ❑ When “pdf” launched...
 - 2048-bit RSA key pair generated
 - Public key on victim machine, private key with the attacker, data encrypted
 - Private key needed to decrypt data
- ❑ Payment of \$400 to decrypt data
 - Could be paid using Bitcoin

CryptoLocker

- ❑ In 2014, “Operation Tovar”
 - Server used by CryptoLocker taken down
 - Private keys recovered
- ❑ How many people paid?
 - Estimates ranged from 0.3% to 41%
 - Appears that actual number was about 1.3%
- ❑ Estimated that bad guys made \$3M

Locky of Ransomware

- ❑ Locky (2016, Russian origin)
 - Email with MS Word document
 - Word doc contained malicious macros
 - User had to enable macros
 - Doc filled with gibberish, “Enable macro if data encoding is incorrect”
- ❑ 2048-bit RSA and 128-bit AES
 - 1 Bitcoin (\$10k at the time)

WannaCry Ransomware

- ❑ WannaCry (2017, North Korean)
 - Worm based on “EternalBlue” exploit
 - EternalBlue supposedly developed by NSA, not revealed to MS for 5 years
- ❑ Asked for \$300 to \$600
- ❑ No data recovered after paying
 - Yet, payments of more than \$130k

WannaCry

- ❑ Infected 200k computers in 150 countries
- ❑ Countries most affected
 - Russia, Ukraine, India, Taiwan
- ❑ Long list of significant organizations affected



Petya Ransomware

- ❑ Petya (2017, origin uncertain)
- ❑ NotPetya (2017, Russian)
 - More advanced variant of Petya using EternalBlue
 - Targeted at Ukraine, but went worldwide
 - Blamed on Russian military
 - Not possible to recover from, even if ransom (\$300) was paid
- ❑ Disruptions caused \$10B in losses

Malware Detection

- ❑ Three common detection methods
 - Signature detection
 - Change detection
 - Anomaly detection

Signature Detection

- ❑ A **signature** may be a string of bits in exe
 - Might also use wildcards, hash values, etc.
- ❑ Advantages
 - Effective on “ordinary” malware
 - Minimal burden for users/administrators
- ❑ Disadvantages
 - Signature file can be large (10s of thousands) making scanning slow
 - Signature files must be kept up to date
 - ***Cannot detect unknown viruses***
 - Cannot detect some advanced types of malware
- ❑ The most popular detection method

Change Detection

- ❑ Viruses must live somewhere
- ❑ If you detect a file has changed, it might have been infected
- ❑ How to detect changes?
 - Hash files and (securely) store hash values
 - Periodically re-compute hashes and compare
 - If hash changes, file **might** be infected

Change Detection

❑ Advantages

- Virtually no false negatives
- Can even detect previously unknown malware

❑ Disadvantages

- Many files change — and often
- Many false alarms (false positives)
- Heavy burden on users/administrators
- If suspicious change detected, then what? Might fall back on signature detection

Anomaly Detection

- ❑ Monitor system for anything “unusual” or “virus-like” or “potentially malicious”...
- ❑ Examples of anomalous things:
 - Files change in some unexpected way
 - System misbehaves in some way
 - Unexpected network activity
 - Unexpected file access
 - ...
- ❑ But, we must first define “normal”
 - And normal can (and must) change over time

Anomaly Detection

❑ Advantages

- Chance of detecting unknown malware

❑ Disadvantages

- No proven track record
- Attacker can make abnormal look normal (go slow)
- Must be combined with another method (e.g., signature detection)

❑ Also popular in intrusion detection (IDS)

❑ Difficult unsolved (unsolvable?) problem

Machine Learning Based Malware Detection

- ❑ Essentially, higher-level “signature”
- ❑ Enables detection of many variants (entire “family”) with one model
- ❑ Many different features considered
- ❑ Many different machine learning and deep learning techniques used

Future of Malware

❑ Trends

- Encrypted, polymorphic, metamorphic
- Recently, botnets and ransomware

❑ The future is bright for malware!

❑ Future of malware detection

- ML/DL based techniques

MISCELLANEOUS SOFTWARE-BASED ATTACKS

Miscellaneous Attacks

- ❑ Numerous attacks involve software
- ❑ We'll discuss a few issues that do not fit into previous categories
 - Salami attack
 - Linearization attack

Salami Attack

❑ What is Salami attack?

- Programmer “slices off” small amounts of money
- Slices are hard for victim to detect

❑ Example

- Bank calculates interest on accounts
- Programmer “slices off” any fraction of a cent and puts it in his own account
- No customer notices missing partial cent
- Bank may not notice any problem
- Over time, programmer makes lots of money!

Salami Attack

- ❑ In LA, four men installed computer chip that overstated amount of gas pumped
 - Customers complained when they had to pay for more gas than tank could hold
 - Hard to detect since chip programmed to give correct amount when 5 or 10 gallons purchased

Linearization Attack

- ❑ Program checks for serial number S123N456
- ❑ For efficiency, check made one character at a time
- ❑ Can attacker take advantage of this?

```
#include <stdio.h>

int main(int argc, const char *argv[])
{
    int i;
    char serial[9]="S123N456\n";

    for(i = 0; i < 8; ++i)
    {
        if(argv[1][i] != serial[i]) break;
    }
    if(i == 8)
    {
        printf("\nSerial number is correct!\n\n");
    }
}
```

Linearization Attack

- ❑ Correct number takes longer than incorrect
- ❑ Attacker tries all 1st characters
 - Find that s takes longest
- ❑ Then she guesses all 2nd characters: S*
- Finds S1 takes longest
- ❑ And so on...
- ❑ Attacker can recover one character at a time!
 - Same principle as used in lock picking

SOFTWARE REVERSE ENGINEERING (SRE)

SRE

❑ Software Reverse Engineering

- Also known as Reverse Code Engineering (RCE)
- Or simply “reversing”

❑ Can be used for **good**...

- Understand malware
- Understand legacy code

❑ ...or **not-so-good**

- Remove usage restrictions from software
- Find and exploit flaws in software
- Cheat at games, etc.

SRE

- ❑ We assume...
 - Reverse engineer is an attacker
 - Attacker only has exe (no source code)
 - No bytecode (i.e., not Java, .Net, etc.)
- ❑ Attacker might want to
 - Understand the software
 - Modify (“patch”) the software
- ❑ SRE usually focused on Windows
 - So we focus on Windows

SRE Tools

❑ Disassembler

- Converts exe to assembly (as best it can)
- Cannot always disassemble 100% correctly
- In general, not possible to re-assemble disassembly into working executable

❑ Debugger

- Must step thru code to completely understand it
- Labor intensive — lack of useful tools

❑ Hex Editor

- To **patch** (modify) exe file

❑ Process Monitor, VMware, etc.

Specific Tools

- ❑ **IDA Pro** — good disassembler/debugger
 - Costs a few hundred dollars (free version exists)
 - Converts binary to assembly (as best it can)
- ❑ **OllyDbg** — high-quality shareware debugger
 - Includes a good disassembler
- ❑ **Hex editor** — to view/modify bits of exe
 - UltraEdit is good — freeware
 - HIEW — useful for patching exe
- ❑ **Process Monitor** — freeware

Why is Debugger Needed?

- ❑ Disassembly gives **static** results
 - Good overview of program logic
 - User must “mentally execute” program
 - Difficult to jump to specific place in the code
- ❑ Debugging is **dynamic**
 - Can set break points
 - Can treat complex code as “black box”
 - And code not always disassembled correctly
- ❑ Disassembly **and** debugging **both** required for any serious SRE task

SRE Necessary Skills

- ❑ Working knowledge of target assembly code
- ❑ Experience with the tools
 - IDA Pro — sophisticated and complex
 - **OllyDbg** — good choice for this class
- ❑ Knowledge of Windows **Portable Executable** (PE) file format
- ❑ Boundless patience and optimism
 - SRE is a tedious and labor-intensive process
 - Often, like finding a needle in a haystack

DIGITAL RIGHTS MANAGEMENT

Digital Rights Management

- ❑ DRM is a good example of limitations of doing security in software
- ❑ We'll discuss
 - What is DRM?
 - A PDF document protection system
 - DRM for streaming media
 - DRM in P2P application
 - DRM within an enterprise

What is DRM?

- ❑ “Remote control” problem
 - Distribute digital content
 - Retain some control on its use, **after delivery**
- ❑ **Digital book** example
 - Digital book sold online could have huge market
 - But might only sell 1 copy!
 - Trivial to make perfect digital copies
 - A fundamental change from pre-digital era
- ❑ Similar comments for digital music, video, etc.

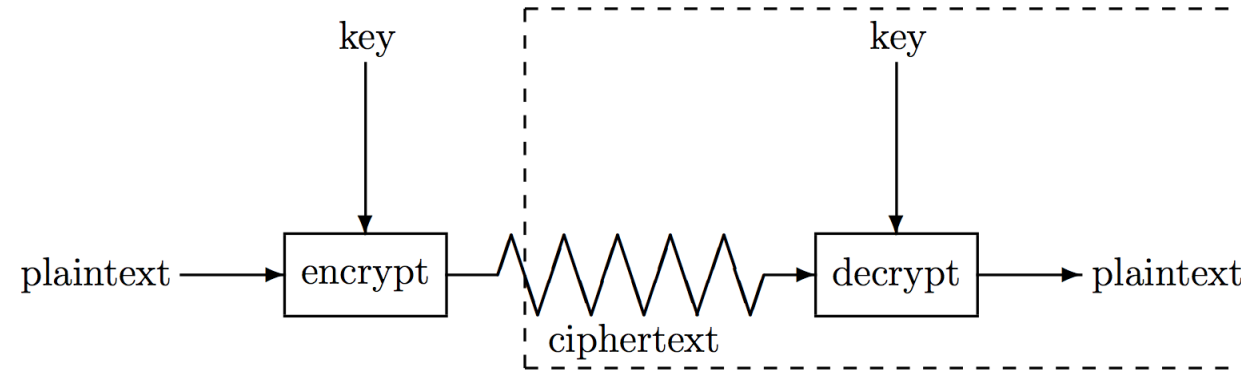
Persistent Protection

- ❑ “Persistent protection” is the fundamental problem in DRM
 - How to enforce restrictions on use of content **after** delivery?
- ❑ Examples of such restrictions
 - No copying
 - Limited number of reads/plays
 - Time limits
 - No forwarding, etc.

What Can be Done?

- ❑ The honor system?
 - Example: Stephen King's, *The Plant*
- ❑ Give up?
 - Internet sales? Regulatory compliance? etc.
- ❑ Lame software-based DRM?
 - The standard DRM system today
- ❑ Better software-based DRM?
 - MediaSnap's goal
- ❑ Tamper-resistant hardware?
 - Closed systems: Game Cube, etc.
 - Open systems: TCG/NGSCB for PCs

Is Crypto the Answer?



- ❑ Attacker's goal is to recover the **key**
- ❑ In standard crypto scenario, attacker has
 - Ciphertext, some plaintext, side-channel info, etc.
- ❑ In DRM scenario, attacker has
 - Everything in the box (at least)
- ❑ Crypto was not designed for this problem!

Is Crypto the Answer?

- ❑ But crypto is necessary
 - To securely deliver the bits
 - To prevent trivial attacks
- ❑ Then attacker will not try to directly attack crypto
- ❑ Attacker will try to find keys in software
 - DRM is “hide and seek” with keys in software!

Current State of DRM

- ❑ At best, **security by obscurity**
 - A derogatory term in security
- ❑ Secret designs
 - In violation of **Kerckhoffs Principle**
- ❑ Over-reliance on crypto
 - “Whoever thinks his problem can be solved using cryptography, doesn’t understand his problem and doesn’t understand cryptography.” —
Attributed by Roger Needham and Butler Lampson to each other

DRM Limitations

❑ The **analog hole**

- When content is rendered, it can be captured in analog form
- DRM **cannot** prevent such an attack

❑ **Human nature** matters

- Absolute DRM security is impossible
- Want something that “works” in practice
- What works depends on context

❑ DRM is not strictly a technical problem!

SECURE SOFTWARE DEVELOPMENT

Penetrate and Patch

- ❑ Usual approach to software development
 - Develop product as quickly as possible
 - Release it without adequate testing
 - Patch the code as flaws are discovered
- ❑ In security, this is “penetrate and patch”
 - A **bad** approach to software development
 - An **even worse** approach to secure software!

Why Penetrate and Patch?

❑ First to market advantage

- First to market likely to become market leader
- Market leader has huge advantage in software
- Users find it safer to “follow the leader”
- Boss won’t complain if your system has a flaw, as long as everybody else has same flaw...
- User can ask more people for support, etc.

❑ Sometimes called “network economics”

Why Penetrate and Patch?

- ❑ Secure software development is hard
 - Costly and time consuming development
 - Costly and time consuming testing
 - Cheaper to let customers do the work!
- ❑ No serious economic disincentive
 - Even if software flaw causes major losses, the software vendor is not liable
 - Is any other product sold this way?
 - Would it matter if vendors were legally liable?

Penetrate and Patch Fallacy

- ❑ **Fallacy:** If you keep patching software, eventually it will be secure
- ❑ Why is this a fallacy?
- ❑ Empirical evidence to the contrary
- ❑ Patches often add new flaws
- ❑ Software is a moving target: new versions, features, changing environment, new uses,...

Open vs Closed Source

- ❑ Open source software
 - The source code is available to user
 - For example, Linux
- ❑ Closed source
 - The source code is not available to user
 - For example, Windows
- ❑ What are the security implications?

Open Source Security

- ❑ Claimed advantages of open source is
 - **More eyeballs:** more people looking at the code should imply fewer flaws
 - A variant on Kerchoffs Principle
- ❑ Is this valid?
 - How many “eyeballs” looking for security flaws?
 - How many “eyeballs” focused on boring parts?
 - How many “eyeballs” belong to security experts?
 - Attackers can also look for flaws!
 - Evil coder might be able to insert a flaw

Open Source Security

- ❑ Open source example: wu-ftp
 - About 8,000 lines of code
 - A security-critical application
 - Was deployed and widely used
 - After 10 years, serious security flaws discovered!
- ❑ More generally, open source software has done little to reduce security flaws
- ❑ Why?
 - Open source follows penetrate and patch model!

Closed Source Security

- ❑ Claimed advantage of closed source
 - Security flaws not as visible to attacker
 - This is a form of “security by obscurity”
- ❑ Is this valid?
 - Many exploits do not require source code
 - Possible to analyze closed source code though it is a lot of work!
 - Is “security by obscurity” real security?

Open vs Closed Source

- ❑ Advocates of open source often cite the **Microsoft fallacy** which states
 1. Microsoft makes bad software
 2. Microsoft software is closed source
 3. Therefore all closed source software is bad
- ❑ Why is this a fallacy?
 - Not logically correct
 - More relevant is the fact that Microsoft follows the penetrate and patch model

Open vs Closed Source

- ❑ No obvious security advantage to either open or closed source
- ❑ More significant than open vs closed source is software development practices
- ❑ Both open and closed source follow the “penetrate and patch” model

Open vs Closed Source

- ❑ If there is no security difference, why is Microsoft software attacked so often?
 - Microsoft is a big target!
 - Attacker wants most “bang for the buck”
- ❑ Few exploits against Mac OS X
 - **Not** because OS X is inherently more secure
 - An OS X attack would do less damage
 - Would bring less “glory” to attacker
- ❑ Next, we consider the theoretical differences

